

Name : Dahiwal Gajanan Manikrao
Roll No : EN24107029
Class : SY - AI \& DS
Subject : Python for Visual AI

Assignment 1

Problem Statement : Convert a color image to grayscale with different methods mentioned below and compare their performances (SSIM, Entropy, PSNR, etc.)

1. Average Method (Arithmetic Mean)
2. Lightness Method
3. Luminosity (Perceptual) Method
4. Luma Method

```
In [ ]: !pip install opencv-python
```

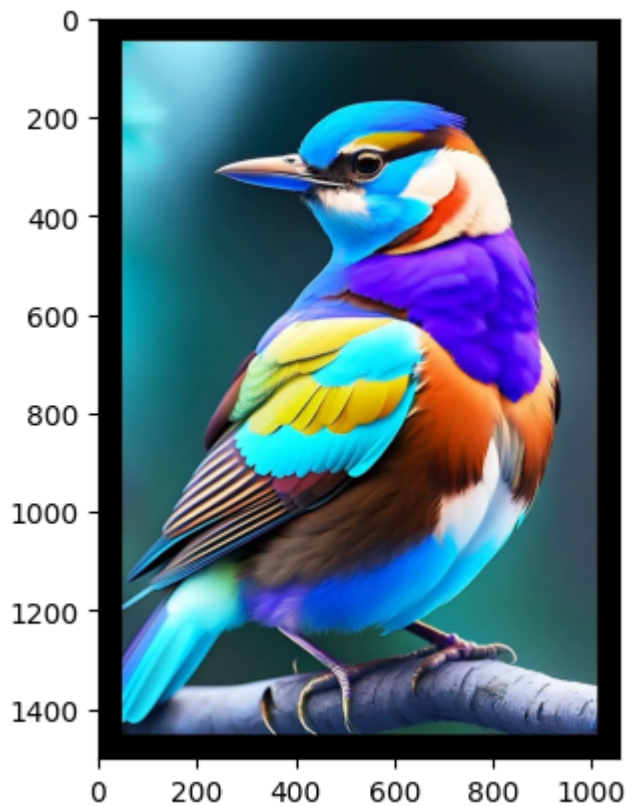
```
Requirement already satisfied: opencv-python in C:\Users\Lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages (4.13.0.92)  
Requirement already satisfied: numpy>=2 in C:\Users\Lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages (from opencv-python) (2.4.2)
```

```
In [ ]: import cv2  
import matplotlib.pyplot as plt  
img = cv2.imread("C:\\Users\\Lenovo\\Downloads\\parrot.webp")
```

```
In [ ]: import numpy as np
```

```
In [ ]: cv2.imwrite('Original.jpg', img, [cv2.IMWRITE_JPEG_QUALITY, 100])  
original_gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)  
plt.imshow(img)
```

```
Out[ ]: <matplotlib.image.AxesImage at 0x28240527750>
```



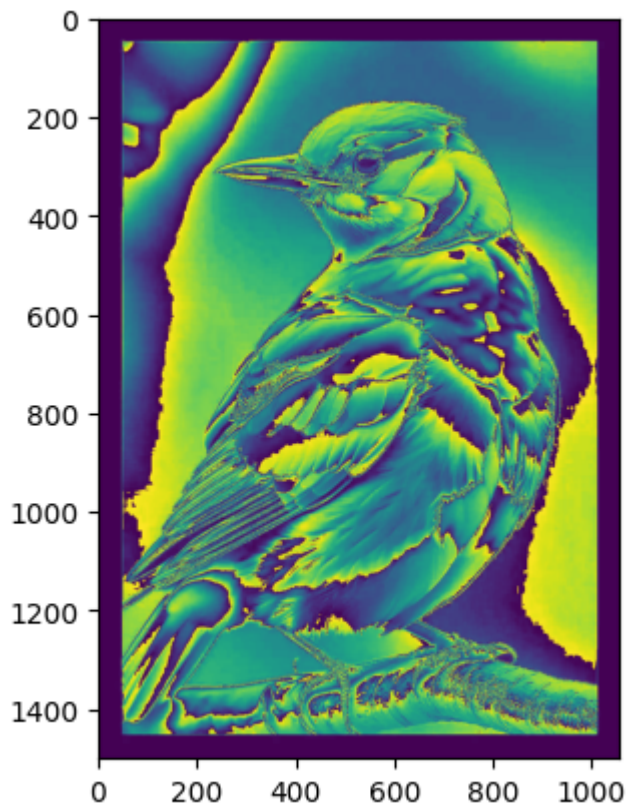
Average Method

$$avg = \frac{R + G + B}{3}$$

Where:

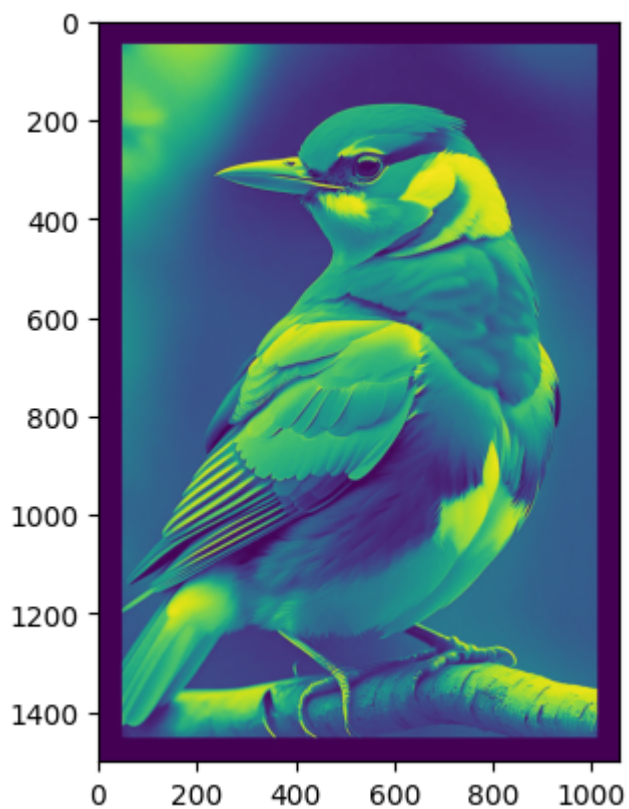
- R, G, B are color intensities
- Gray is the resulting grayscale value

```
In [ ]: r, g, b = cv2.split(img)
        avg = (r + g + b) / 3
        gray_avg = plt.imshow(avg)
```



```
In [ ]: import numpy as np
img_1 = img.astype(np.float64)
r , g, b = cv2.split(img_1)
avg2 = (r+g+b)/3
cv2.imwrite('gray_avg.jpg', avg2, [cv2.IMWRITE_JPEG_QUALITY, 100])
plt.imshow(avg2)
```

Out[]: <matplotlib.image.AxesImage at 0x28203a751d0>



Luminosity Method

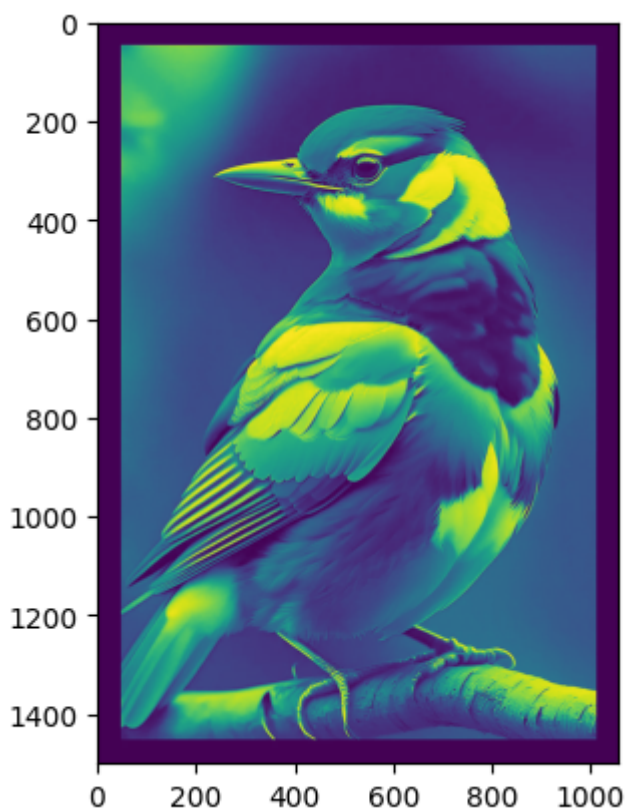
$$\textit{luminosity} = 0.299R + 0.5872G + 0.144B$$

Where:

- R, G, B are color channels
- Coefficients represent human eye sensitivity

```
In [ ]: # Luminosity Method :  
  
luminosity = (0.299*r) + (0.587*g) + (0.114*b)  
cv2.imwrite('Gray_Luminosity.jpg', luminosity, [cv2.IMWRITE_JPEG_QUALITY,  
plt.imshow(luminosity)
```

```
Out[ ]: <matplotlib.image.AxesImage at 0x28203ce2350>
```



Luma Transformation

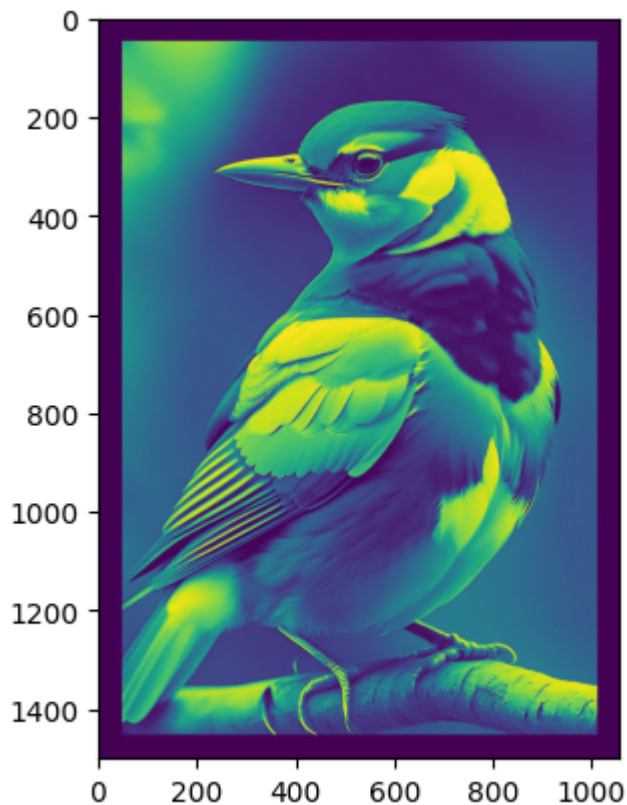
$$Y' = 0.2126R + 0.7152G + 0.0722B$$

Where:

- R, G, B are the red, green, and blue components
- Y' represents the luma (perceived brightness)

```
In [ ]: # Luma method  
  
luma = (0.2126*r) + (0.7152*g) + (0.0722*b)  
cv2.imwrite('Gray_luma.jpg', luma, [cv2.IMWRITE_JPEG_QUALITY, 100])  
plt.imshow(luma)
```

Out[]: <matplotlib.image.AxesImage at 0x28203d64f50>



Lightness Method

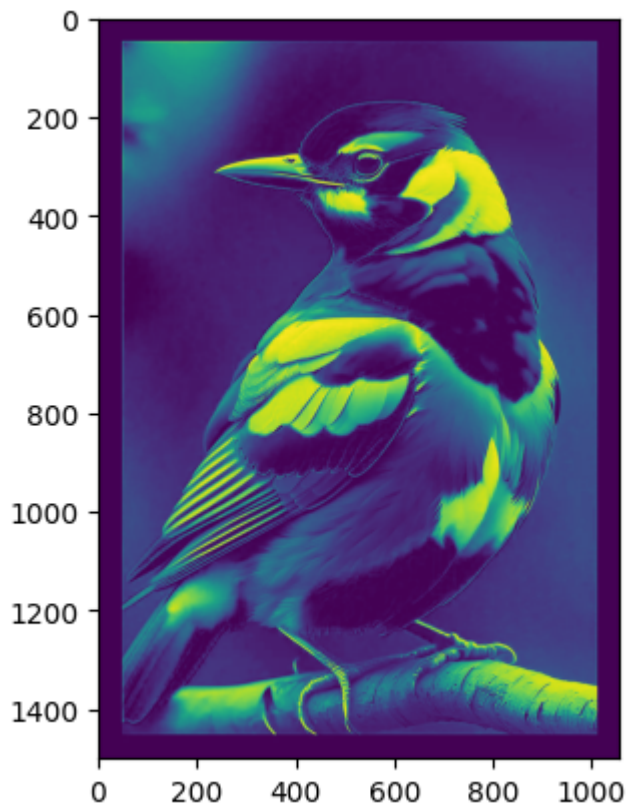
$$lightness = \frac{\max(R, G, B) + \min(R, G, B)}{2}$$

Where:

- $\max(R, G, B)$ is the maximum intensity
- $\min(R, G, B)$ is the minimum intensity

```
In [ ]: # Lightness Method :  
  
Max_index = cv2.max(r,g,b)  
Min_index = cv2.min(r,g,b)  
lightness = (Max_index + Min_index)/2  
  
cv2.imwrite('Gray_light.jpg', lightness, [cv2.IMWRITE_JPEG_QUALITY, 100])  
plt.imshow(lightness)
```

Out[]: <matplotlib.image.AxesImage at 0x28203db3c50>



Mean Squared Error (MSE)

$$MSE = \frac{1}{MN} \sum (I - K)^2$$

Where:

- I = original image
- K = processed image
- M, N = image dimensions

Peak Signal-to-Noise Ratio (PSNR)

$$PSNR = 10 \cdot \log_{10} \left(\frac{MAX^2}{MSE} \right)$$

Where:

- MAX = maximum pixel value (255)
- MSE = Mean Squared Error between original and grayscale image

```
In [ ]: import math
```

```
In [ ]: from math import sqrt, log10
```

```
In [ ]: img1 = original_gray  
img2 = avg  
img3 = luminosity  
img4 = luma  
img5 = lightness
```

```
In [ ]: def calculate_psnr(img1, img2):
        img1 = img1.astype(np.float64)
        img2 = img2.astype(np.float64)

        mse = np.mean((img1 - img2) ** 2)

        if mse == 0:
            return 100

        return 10 * math.log10((255 * 255) / mse)

print("Original and Avg:", calculate_psnr(img1, img2))
print("Original and Luminosity:", calculate_psnr(img1, img3))
print("Original and Luma:", calculate_psnr(img1, img4))
print("Original and Light:", calculate_psnr(img1, img5))
```

```
Original and Avg: 10.268902439761625
Original and Luminosity: 26.983960328317657
Original and Luma: 30.244128197666093
Original and Light: 14.36962430912531
```

Entropy

$$H = - \sum p(i) \log_2 p(i)$$

Where:

- $p(i)$ = probability of intensity i
- H = information content

Entropy measures the amount of information in an image.

- High entropy → more details
- Low entropy → less information

```
In [ ]: from skimage.metrics import structural_similarity as ssim
import skimage
Original_Entropy = skimage.measure.shannon_entropy(img1)
print(f"Original Entropy : {Original_Entropy}")
Avg_Entropy = skimage.measure.shannon_entropy(img2)
print(f"Avg Entropy : {Avg_Entropy}")
Luminosity_Entropy = skimage.measure.shannon_entropy(img3)
print(f"Luminosity Entropy : {Luminosity_Entropy}")
Luma_Entropy = skimage.measure.shannon_entropy(img4)
print(f"Luma Entropy : {Luma_Entropy}")
Light_Entropy = skimage.measure.shannon_entropy(img5)
print(f"Light Entropy : {Light_Entropy}")
```

```
Original Entropy : 7.189153577996808
Avg Entropy : 7.449420151360743
Luminosity Entropy : 12.784411985651099
Luma Entropy : 12.817635245296222
Light Entropy : 6.713896914637987
```

Structural Similarity Index (SSIM)

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

Where:

- μ_x, μ_y = mean intensity
- σ_x, σ_y = variance
- σ_{xy} = covariance

SSIM measures structural similarity between two images.

- Value ranges from -1 to 1
- Higher value → better quality

```
In [ ]: score = ssim(img1, img2, data_range=255)
        print(f"SSIM of Original and Average is : {score}")
```

SSIM of Original and Average is : 0.6261634138929434

```
In [ ]: score2 = ssim(img1, img3, data_range=255)
        print(f"SSIM of Original and Luminosity is : {score2}")
```

SSIM of Original and Luminosity is : 0.9832216163815775

```
In [ ]: score3 = ssim(img1, img4, data_range=255)
        print(f"SSIM of Original and Luma is : {score3}")
```

SSIM of Original and Luma is : 0.9844686822225568

```
In [ ]: score4 = ssim(img1, img5, data_range=255)
        print(f"SSIM of Original and Lightness is : {score4}")
```

SSIM of Original and Lightness is : 0.7742757605372043

Conclusion

In this Experiment, We have performed 4 methods to convert the Colored image into grayscale image Average, Luminosity, Luma, Lightness. After comparing this methods using PSNR, SSIM and Entropy we can say that Luma is the best method to convert Colored image to GrayScale Image.

```
In [ ]:
```